Dockerised Healthcare Planning Assistant Agent

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Mayuri Patidar	EN22CS301593
2.	Meet Solanki	EN22CS301595
3.	Mokshika Yadav	EN22CS301614
4.	Nirdesh Sharma	EN22CS301655
5.	Palak Shahdadpuri	EN22CS301677
6.	Pankaj	EN22CS301679

Group Name: 11D6

Project Number: DO-22

Industry Mentor Name: Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2026

1. PROBLEM STATEMENT AND OBJECTIVES

1.1 PROBLEM STATEMENT

Healthcare awareness and early symptom assessment play a critical role in preventing serious medical conditions and enabling timely treatment. However, in many situations individuals are unable to immediately access professional medical consultation due to factors such as time constraints, geographical limitations, long waiting times at healthcare facilities, or lack of awareness about the seriousness of their symptoms. As a result, people often rely on unreliable internet searches or self-diagnosis methods, which can lead to misinformation, unnecessary anxiety, or delayed medical care.

With the increasing advancement of Artificial Intelligence and Large Language Models (LLMs), it has become possible to develop intelligent systems that can assist users in understanding their symptoms and receiving preliminary health guidance. These systems can analyze user-provided information such as symptoms, age, and other demographic factors, and generate structured suggestions that help individuals take informed next steps regarding their health. By processing user inputs and presenting information in a clear format, such systems help users better interpret their health conditions and reduce confusion caused by scattered or unreliable online sources.

Despite the availability of many healthcare websites and symptom-checker tools, many existing solutions suffer from several limitations. Many platforms have complex user interfaces that make them difficult to use, especially for individuals who are not technologically experienced. In addition, the information provided is often not well organized, making it challenging for users to clearly understand the results. The lack of personalized guidance and structured explanations further reduces the effectiveness of these systems, as users may still struggle to interpret the medical information provided to them.

To address these challenges, there is a need for a simple, accessible, and intelligent web-based application that allows users to input their symptoms and receive structured guidance in an understandable format. Such a system should focus on presenting information in a clear and organized manner so that users can easily interpret the results. By combining modern web technologies with advanced AI models, it is possible to create a tool that supports users in gaining preliminary insights into their health conditions.

The Healthcare Planning Assistant project aims to solve this problem by developing a web-based application built using the FastAPI framework and integrated with Large Language Models through LangChain. The application allows users to enter their symptoms and basic demographic information through an intuitive interface. The system then processes the information and generates structured health-related guidance that helps users better understand their health situation.

The goal of this system is not to replace professional medical consultation but to act as a supportive tool that helps users better understand their symptoms and encourages them to take appropriate action when necessary. By providing preliminary health insights in a structured and user-friendly format, the application aims to improve healthcare awareness and assist users in making more informed health decisions.

The system primarily focuses on providing the following types of guidance:

1. **Possible Conditions** – Potential health conditions that may be related to the symptoms entered by the user.
2. **Precautions** – Preventive measures or steps that users should consider to avoid worsening of symptoms.
3. **Medication Guidance** – General suggestions regarding medications that may commonly be used for certain symptoms.
4. **Nutrition Advice** – Dietary recommendations that may help support recovery and maintain overall health.

1.2 PROJECT OBJECTIVES

The primary objective of the Healthcare Planning Assistant project is to design and develop an intelligent web-based system that can provide users with structured healthcare guidance based on their reported symptoms and basic demographic information. The system leverages modern web development frameworks and artificial intelligence technologies to generate informative and organized responses that assist users in understanding potential health conditions and recommended precautions.

One of the key objectives of the project is to develop an easy-to-use interface that enables users to input their symptoms and personal details without requiring any technical knowledge. The application should be accessible, responsive, and capable of guiding users through a simple workflow from symptom submission to receiving healthcare advice. By ensuring simplicity and usability, the system aims to make healthcare-related information more accessible to a wide range of users.

Another important objective is to integrate Large Language Models (LLMs) into the system to enable intelligent processing of user input. By utilizing language models through LangChain, the system can generate meaningful responses based on the symptoms provided by the user. This allows the application to produce structured outputs such as possible health conditions, recommended precautions, medication-related guidance, and nutritional advice.

The project also aims to implement a modular and scalable architecture using modern backend technologies such as FastAPI and Pydantic. This ensures that the system remains maintainable, well-structured, and easily extendable for future enhancements such as adding persistent data storage, advanced analytics, or additional healthcare modules.

Furthermore, the project seeks to demonstrate the practical application of AI-powered services in healthcare support systems. By integrating APIs such as Google Gemini and Groq LLM models, the application highlights how artificial intelligence can be used to build intelligent assistants that provide useful insights in real-world scenarios. This approach also illustrates how AI technologies can enhance digital healthcare tools and improve the accessibility of medical information.

Another objective of the project is to ensure that the generated healthcare information is organized into clearly defined sections. Presenting information in structured categories improves readability and helps users better understand the results generated by the system.

The key objectives of the system include:

1. Designing a user-friendly web interface for entering symptoms and personal details.
2. Integrating Large Language Models through LangChain to process user inputs intelligently.
3. Generating structured healthcare guidance based on symptoms and demographic information.
4. Implementing a modular and scalable backend architecture using FastAPI and Pydantic.

Finally, the project aims to provide a proof-of-concept solution that showcases the potential of combining web technologies with artificial intelligence to build interactive healthcare assistance platforms. The system serves as a foundation for future developments where more advanced healthcare features, machine learning models, or data storage mechanisms can be integrated.

The long-term goals of the project include:

1. Demonstrating the practical use of AI-powered healthcare assistants.
2. Improving health awareness and early symptom understanding among users.
3. Providing a foundation for future expansion with additional healthcare modules and analytics.
4. Encouraging the development of AI-integrated healthcare support systems for real-world applications.

1.3 SCOPE OF THE PROJECT

The scope of the Healthcare Planning Assistant project focuses on developing a web-based application that provides preliminary healthcare guidance based on user-reported symptoms and demographic details. The system is designed to assist users in gaining a better

understanding of their health conditions by generating structured advisory responses using artificial intelligence.

The project primarily covers the design and implementation of a backend service using the FastAPI framework, which handles user requests, processes form submissions, and communicates with Large Language Models to generate responses. The application also includes a frontend user interface built using Jinja2 templates and Bootstrap styling, allowing users to interact with the system through a simple and visually organized web form.

Within the scope of the project, the system collects information such as the user's symptoms, age, gender, and the duration of symptoms. Based on this information, the backend application sends the data to a selected AI model provider such as Google Gemini or Groq through LangChain. The AI model then generates healthcare-related guidance which is structured into four main sections: possible conditions, precautions, medication guidance, and nutrition advice.

The system also includes input validation mechanisms to ensure that user-provided information meets the required criteria. For example, the age must fall within a valid range, the symptoms field cannot be empty, and the gender value must match predefined options. These validations help maintain the reliability of the generated responses.

However, the current scope of the project intentionally excludes certain advanced functionalities. The application does not store user data in a database, meaning that no historical records or persistent data storage are maintained. Additionally, the system does not provide actual medical diagnoses or replace professional healthcare consultation. Instead, it functions as an informational assistant designed to offer general guidance and awareness.

The scope of the project includes the following key aspects:

1. Development of a web-based healthcare assistance application.
2. Collection of user symptoms and demographic information through a web interface.
3. Integration with Large Language Models using LangChain for generating responses.
4. Presentation of structured healthcare guidance including possible conditions, precautions, medication guidance, and nutrition advice.

The scope is limited to generating advisory content using external AI models and presenting the results to the user through the web interface. Advanced features such as machine learning model training, electronic health record integration, predictive analytics, or hospital management systems are outside the current scope of this project.

Despite these limitations, the project successfully demonstrates how modern AI technologies and web frameworks can be combined to create an intelligent healthcare assistance platform.

The current implementation serves as a foundational system that can be expanded in the future to include additional healthcare capabilities, improved analytics, and more sophisticated AI-based medical support tools.

2. PROPOSED SOLUTION

The proposed solution for the identified problem is the development of an intelligent web-based application called Healthcare Planning Assistant. The application is designed to provide users with structured healthcare guidance based on the symptoms they enter along with basic demographic information such as age and gender. The system leverages modern web technologies and Large Language Models (LLMs) to analyze the input data and generate useful health-related insights.

The primary goal of the proposed solution is to create a simple and accessible platform where users can quickly obtain preliminary healthcare guidance without navigating through complex medical resources. By combining the capabilities of modern backend frameworks with advanced AI-powered language models, the system can process natural language symptom descriptions and produce meaningful responses in a structured format.

The Healthcare Planning Assistant is implemented as a web application using the FastAPI framework for backend development. FastAPI is known for its high performance, asynchronous capabilities, and built-in support for data validation using Pydantic models. The frontend of the system uses Jinja2 templates and Bootstrap styling, which allows the application to present an interactive and user-friendly interface.

The system interacts with external AI services such as Google Gemini and Groq language models through the LangChain framework. LangChain acts as an orchestration layer that manages prompt templates, model communication, and response parsing. When a user submits their symptoms, the application sends this information to the selected AI model, which generates healthcare guidance in natural language form.

The generated output is then processed and organized into clearly defined sections such as possible health conditions, precautions, medication guidance, and nutrition advice. These structured results are displayed to the user through the web interface, allowing them to easily understand the suggested information.

The proposed solution mainly includes the following components:

1. A web-based interface that allows users to enter symptoms and demographic information.
2. A FastAPI backend system that processes requests and manages application logic.

3. Integration with AI language models through LangChain to generate healthcare guidance.
4. Presentation of results in structured sections such as possible conditions, precautions, medication guidance, and nutrition advice.

This solution focuses on delivering a clean and organized workflow that integrates frontend interaction, backend processing, and AI-powered advisory generation. The system is modular in design, making it easy to extend in the future with additional features such as database storage, analytics dashboards, or integration with healthcare services.

Overall, the proposed solution demonstrates how modern AI technologies and web development frameworks can be used together to create an intelligent healthcare support system that provides informative and structured guidance to users.

2.1 KEY FEATURES

The Healthcare Planning Assistant application includes several important features that enhance its functionality and usability. These features are designed to provide users with a simple interface while ensuring efficient backend processing and AI-based response generation.

Some of the key features of the system are listed below:

1. **Symptom Input Interface**
   The application provides an intuitive form where users can enter their symptoms along with demographic details such as age, gender, and symptom duration.
2. **AI-Powered Health Guidance**
   The system uses advanced Large Language Models to analyze the symptoms and generate health-related advisory content.
3. **Multiple Model Support**
   Users can select between different AI model providers such as Google Gemini or Groq LLM for generating responses.
4. **Structured Response Format**
   The generated healthcare advice is presented in four structured sections to make the output easy to read and understand.
5. **Input Validation**
   The system validates user input using Pydantic models to ensure data consistency and accuracy before processing.
6. **Server-Rendered User Interface**
   The application uses Jinja2 templates to dynamically render results and display them on the same page after submission.

7. **Error Handling Mechanism**
   The application provides user-friendly error messages in case of invalid input, configuration issues, or LLM service failures.
8. **Lightweight and Scalable Architecture**
   The system is built with a modular architecture that allows easy extension and maintenance.

2.2 OVERALL ARCHITECTURE / WORKFLOW

2.2.1 SYSTEM ARCHITECTURE

The Healthcare Planning Assistant follows a layered architecture, which separates different responsibilities within the system. This design ensures that each component performs a specific role while maintaining clear communication between layers.

The architecture of the system can be divided into the following layers:

1. Presentation Layer (User Interface)

The presentation layer is responsible for interacting with the user. It consists of HTML templates built using the Jinja2 templating engine and styled with Bootstrap. The user interface contains a form where users can enter their symptoms, age, gender, duration of symptoms, and select the AI model provider.

When the user submits the form, the request is sent to the backend server for processing. The results returned by the backend are displayed on the same webpage as structured information cards.

2. Web/API Layer

The Web/API layer is implemented using the FastAPI framework. This layer handles incoming HTTP requests from the user interface and routes them to the appropriate backend functions.

Two main endpoints are defined in the system:

GET / – Loads the main webpage containing the symptom input form.
POST /analyze – Receives the submitted form data and triggers the backend analysis process.

This layer also performs basic input validation and ensures that required fields such as symptoms, age, and gender are correctly provided by the user.

3. Domain and Data Validation Layer

The domain layer contains Pydantic models that define the structure of the data used within the system. These models ensure that the input data follows the required format and constraints.

For example, the system validates that:

1.Age falls within a valid range
2. Symptoms field is not empty
3.Gender matches one of the allowed values
4. Model selection is valid

By enforcing these validations, the system prevents incorrect data from being processed and improves the reliability of the application.

4. Service Layer

The service layer acts as the core processing unit of the application. It is responsible for orchestrating the interaction between the backend system and the external AI models.

This layer includes an LLM Orchestrator, which determines which AI provider to use based on the user's selection. If the user selects Gemini, the request is sent to the Gemini client. If the user selects Groq, the request is forwarded to the Groq client.

The service layer also manages prompt creation and response parsing.

5. AI Model Interaction Layer

This layer communicates directly with the external AI services using the LangChain framework. LangChain provides tools for building prompt templates, connecting to chat models, and processing their responses.

The system sends the following information to the AI model:

1.Symptoms entered by the user
2. Age of the user
3. Gender of the user
4.Duration of symptoms

The AI model analyzes the information and generates a detailed textual response.

6. Response Parsing and Output Generation

Once the response is received from the AI model, the application processes the text and extracts the relevant sections. The response is structured into four main categories:

1.Possible Health Conditions
2. Precautions
3.Medication Guidance
4.Nutrition Advice

If any section is missing, the system automatically fills in default information to ensure the output remains complete.

The structured data is then returned to the user interface for display.

Workflow Summary

The overall workflow of the system can be summarized in the following steps:

1. The user opens the application homepage in the browser.
2. The user enters symptoms and demographic details into the form.
3. The form is submitted to the FastAPI backend.
4. The backend validates the input using Pydantic models.
5. The system selects the appropriate AI model through the LLM orchestrator.
6. The symptom data is sent to the selected AI provider.
7. The AI model generates healthcare guidance.
8. The backend parses and structures the output.
9. The structured results are rendered on the webpage and shown to the user.

2.2.2 DATA FLOW DIAGRAM(DFD)

A Data Flow Diagram (DFD) represents the flow of data within a system. It shows how data moves from input to processing and finally to output. It also represents processes, data stores, and external entities.

In this system, the DFD explains how user input (symptoms and details) is processed and how an AI-based healthcare response is generated and displayed.

Description:

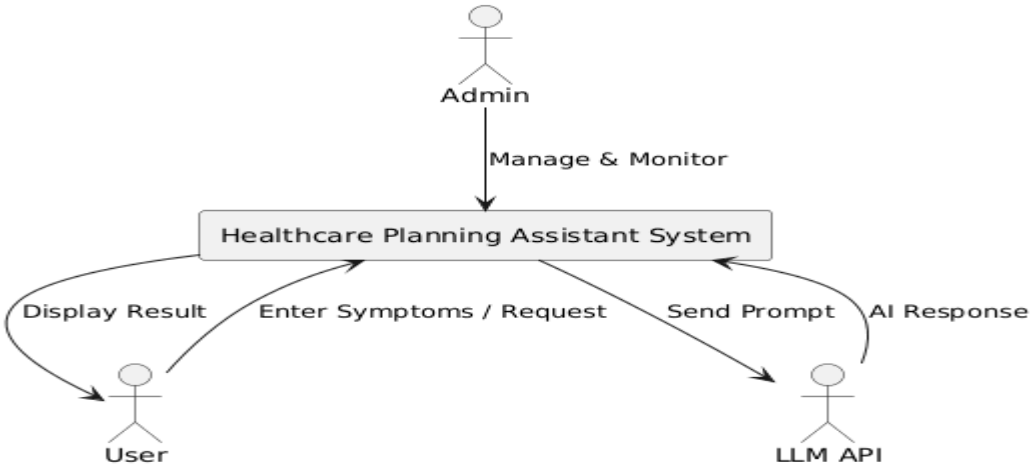
1. It provides a high-level overview of the entire system
2. The system is represented as a single process
3. It shows interaction between external entities and the system

Main Components:

1. **User**
    - Sends symptoms and request to the system
    - Receives result from the system
2. **Healthcare Planning Assistant System**

- Processes the request and generates response

3. **LLM API (External System)**
   - Provides AI-generated response
4. **Admin**
   - Manages and monitors the system



2.2.3 USE CASE DIAGRAM

A Use Case Diagram represents the interaction between users (actors) and the system. It shows the different functionalities (use cases) provided by the system and how users interact with them.

In this system, the Use Case Diagram explains how a user interacts with the Healthcare Planning Assistant System to input symptoms and receive AI-based healthcare guidance. It also shows the role of the admin and external AI service.

Main Use Cases in the System:

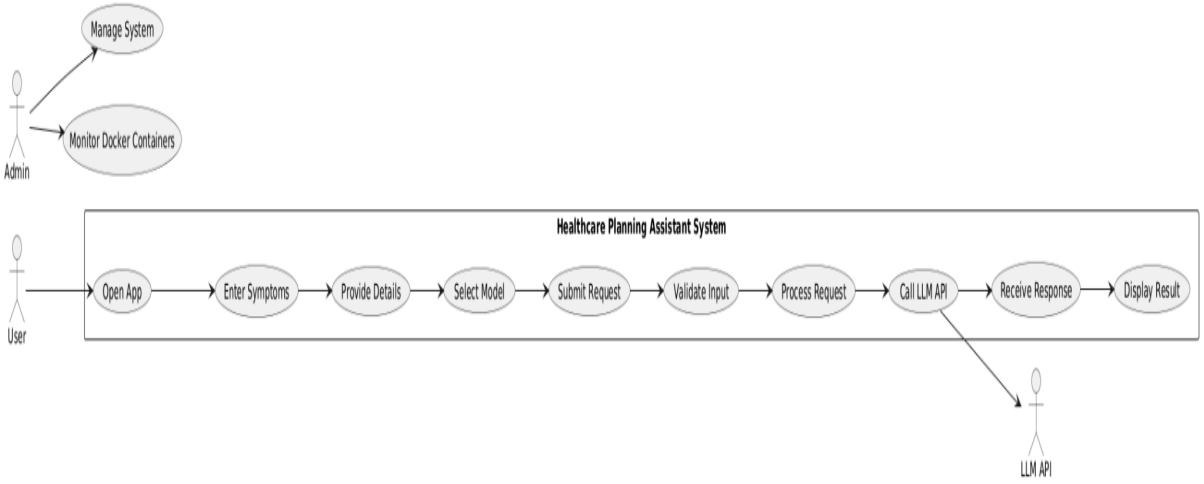
1. Open Application
2. Enter Symptoms
3. Provide Details (age, gender, duration, etc.)
4. Select Model
5. Submit Request
6. Validate Input
7. Process Request

8. Call LLM API
9. Receive Response

10. Display Result

Actors in the System:

1. **User -** Interacts with the system to enter symptoms and get results
2. **Admin -** Manages the system and monitors Docker containers
3. **LLM API (External System) -** Provides AI-generated responses



2.2.4 CLASS DIAGRAM

A Class Diagram represents the structure of a system. It shows the system's classes, their attributes (data), methods (functions), and the relationships between them. It helps in understanding how different components of the system interact with each other.

In this system, the Class Diagram explains how user symptom data is processed using an AI model and how a structured advisory response is generated.

Main Classes in the System:

1. SymptomsRequest

1. Stores user input data
2. Attributes:
    1. symptoms : String
    2. age : int
    3. sex : String
    4. duration : String
    5. model : String
3. This class sends the request to the system for processing

2. LLMOrchestrator

1. Acts as the main controller of the system
2. Method:- processRequest(req: SymptomsRequest): AdvisoryResponse
3. It manages the complete flow and coordinates between components

3. LLMClient

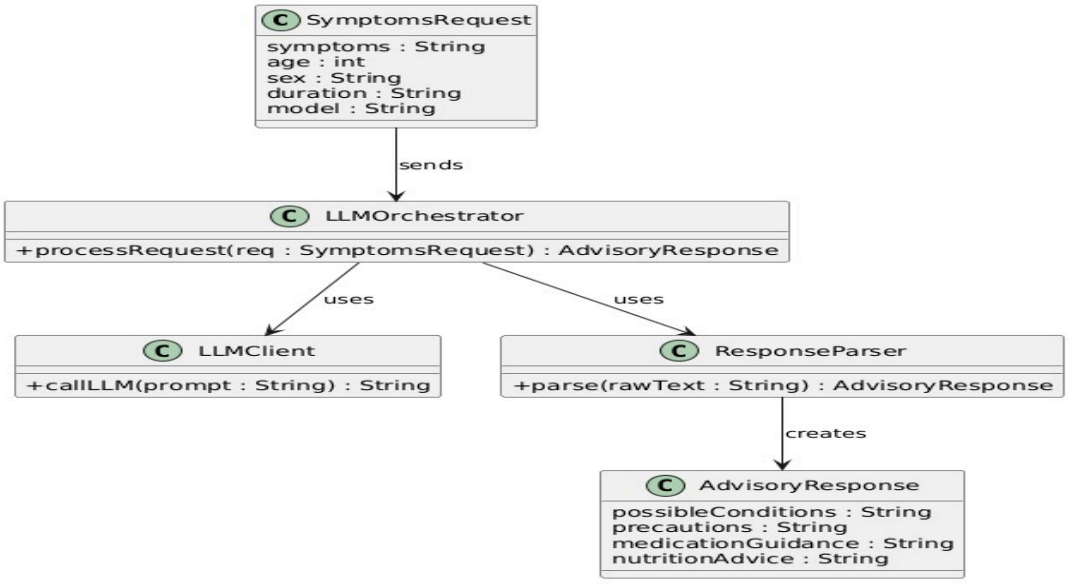
1. Responsible for communicating with the AI model
2. Method:- callLLM(prompt: String): String
3. It sends request to AI and receives raw response

4. ResponseParser

1. Processes the raw AI response
2. Method:- parse(rawText: String): AdvisoryResponse
3. It converts unstructured AI output into structured format

5. AdvisoryResponse

1. Stores the final structured result
2. Attributes:
    1. possibleConditions : String
    2. precautions : String
    3. medicationGuidance : String
    4. nutritionAdvice : String
3. This is the final output shown to the user



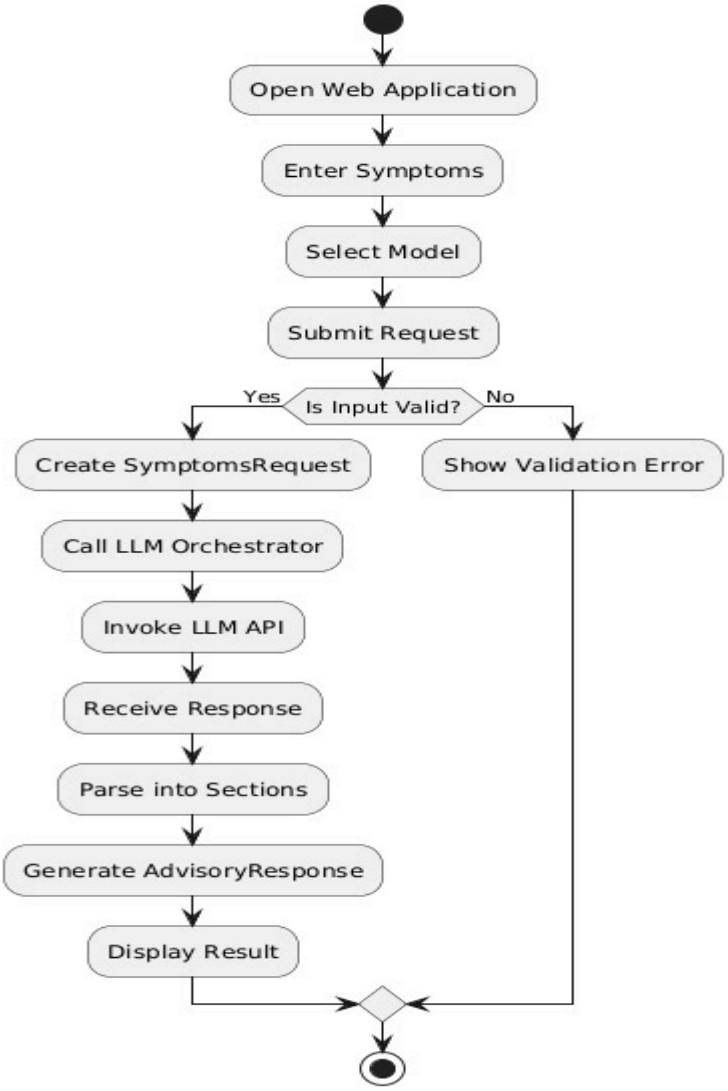
2.2.5 ACTIVITY DIAGRAM

An Activity Diagram represents the workflow of a system. It shows the sequence of activities, decision points, and the flow of control from start to end. It helps in understanding how the system performs tasks step-by-step.

In this system, the Activity Diagram explains how user input (symptoms or problem details) is processed and how an AI-based response is generated and displayed to the user.

Main Activities in the System:

1. Start
2. Open Web Application
3. Enter Symptoms
4. Select Model
5. Submit Request
6. Validate Input
7. Decision (Input Valid?)
    a. If **No** → Show Validation Error → End
    b. If **Yes** → Continue processing
8. Create Symptoms Request
9. Call LLM Orchestrator
10. Invoke LLM API
11. Receive Response
12. Parse Response into Sections
13. Generate Advisory Response
14. Display Result
15. End



2.3 TOOLS & TECHNOLOGIES USED

The development of the Healthcare Planning Assistant involves several tools and technologies that enable efficient application development, AI integration, and user interaction.

1. **Programming Language -** Python is used as the primary programming language for developing the backend system due to its simplicity, extensive libraries, and strong support for AI and web development.
2. **Backend Framework -** FastAPI is used to build the backend web application. It provides high performance, asynchronous request handling, and automatic validation using Pydantic models.

3. **Frontend Technologies -** The frontend interface is built using the following technologies:

   HTML for creating the structure of the web pages
   Jinja2 for dynamic template rendering
   Bootstrap for responsive styling and UI components

   These technologies ensure that the user interface is both interactive and visually organized.

4. **AI Integration Framework -** LangChain is used to connect the application with Large Language Models. It provides tools for prompt templates, model chaining, and response parsing.
5. **Large Language Models -** The system supports the following AI model providers:

   Google Gemini API – used for generating intelligent health guidance responses.
   Groq LLM API – provides an alternative model for generating responses.

6. **Data Validation -** Pydantic is used for defining data models and validating user inputs before processing.
7. **Environment Configuration -** The application uses environment variables stored in a .env file for securely managing API keys and configuration settings.
8. **Web Server -** The application runs on an ASGI server such as Uvicorn, which enables asynchronous request handling and fast performance.
9. **Development Tools -** Some additional tools used during development include:
   Code editors such as Visual Studio Code
   Version control using Git
   Dependency management using Python virtual environments

3. RESULTS AND OUTPUT

The implementation of the Healthcare Planning Assistant resulted in a functional web-based system capable of accepting user symptom inputs and generating structured healthcare guidance using Large Language Models. The application successfully integrates a user-friendly interface with a backend processing system that communicates with AI model providers to generate advisory responses.

The system was tested using various symptom inputs to evaluate the accuracy, reliability, and responsiveness of the application. The results demonstrate that the application is capable of accepting user input, validating the data, interacting with external AI models, and displaying structured healthcare advice in a clear and organized format.

The generated outputs are divided into four major sections: possible health conditions, precautions, medication guidance, and nutrition advice. These sections help users understand potential causes of their symptoms and take appropriate preliminary steps regarding their health.

The following subsections describe the output results obtained from the system along with explanations of the user interface and system responses.

3.1 SCREENSHOTS / OUTPUT

This section presents screenshots of the application interface and the results generated after user interaction. These screenshots illustrate how the system operates from the user's perspective and demonstrate the functionality of the implemented features.

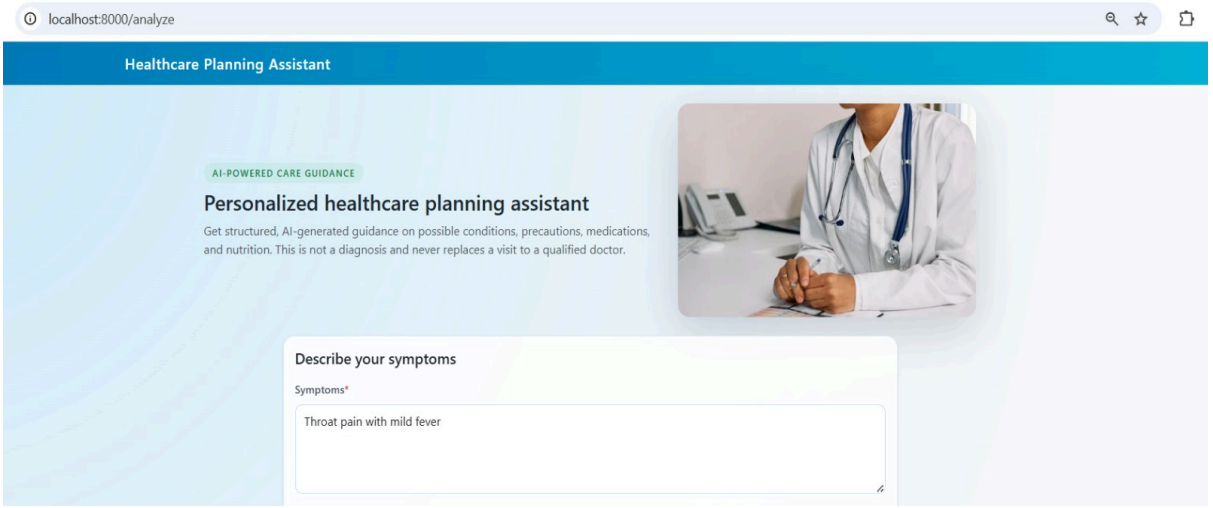
3.1.1 Homepage Interface

The homepage of the Healthcare Planning Assistant displays a clean and organized form where users can enter their symptoms and demographic details. The interface is designed to be simple and easy to use so that individuals without technical knowledge can interact with the system effortlessly.

The form includes the following input fields:

1. Symptoms text area where users describe the symptoms they are experiencing
2. Age input field
3. Gender selection option
4. Duration of symptoms input field
5. Model selection option (Gemini or Groq)

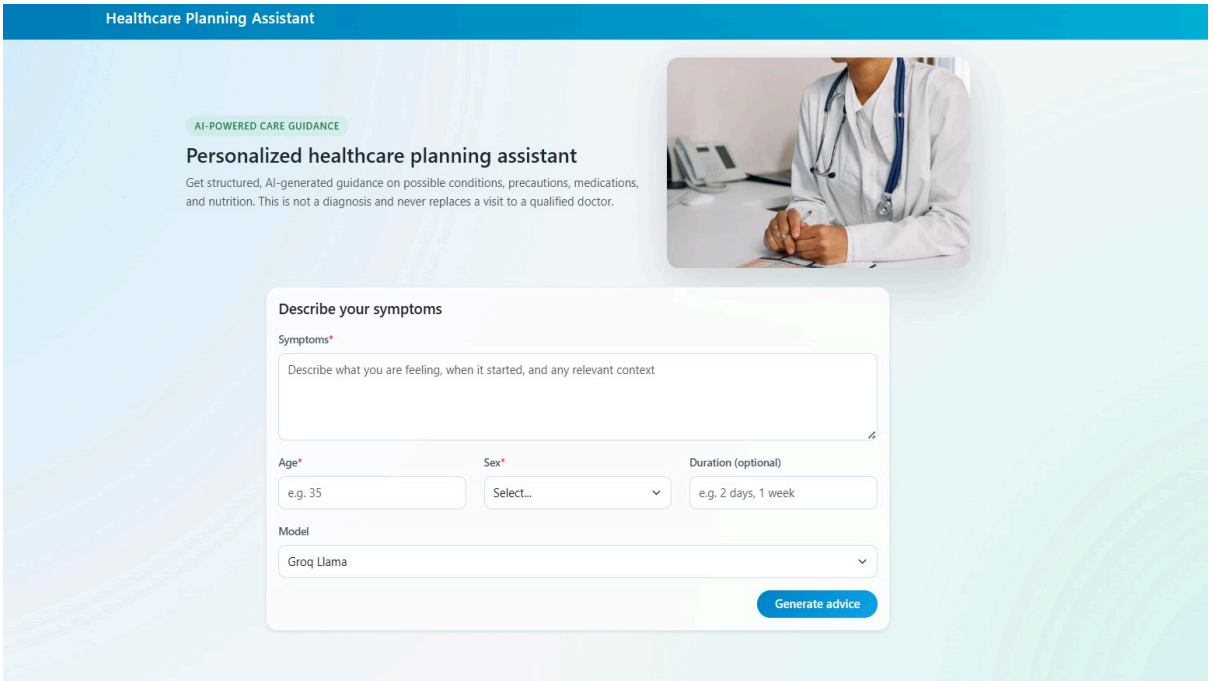
After filling in the required details, users can submit the form to initiate the analysis process.



3.1.2 Symptom Submission Example

In this example, the user enters symptoms such as fever, headache, and body ache along with their age, gender, and duration of symptoms. After submitting the form, the backend processes the request and sends the information to the selected AI model.

The application validates the inputs before sending the request to the AI model to ensure that all required fields are correctly provided.



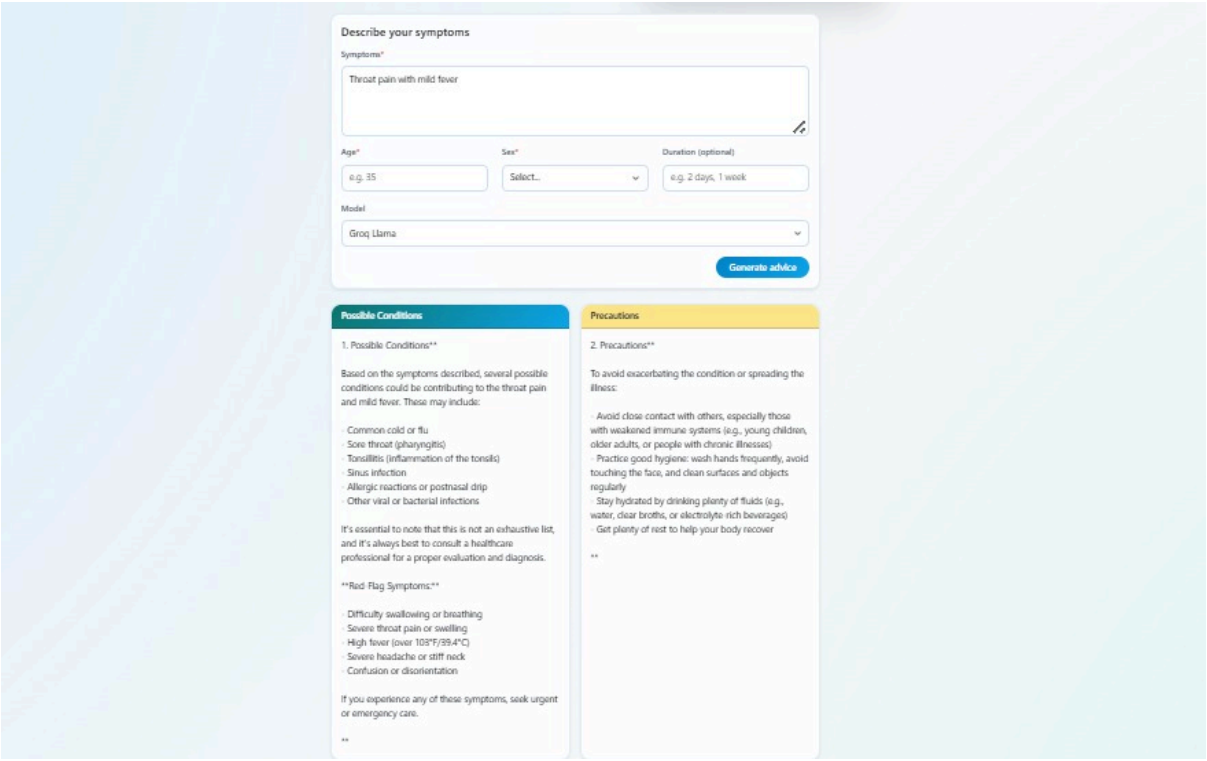
3.1.3 Generated Healthcare Advisory Output

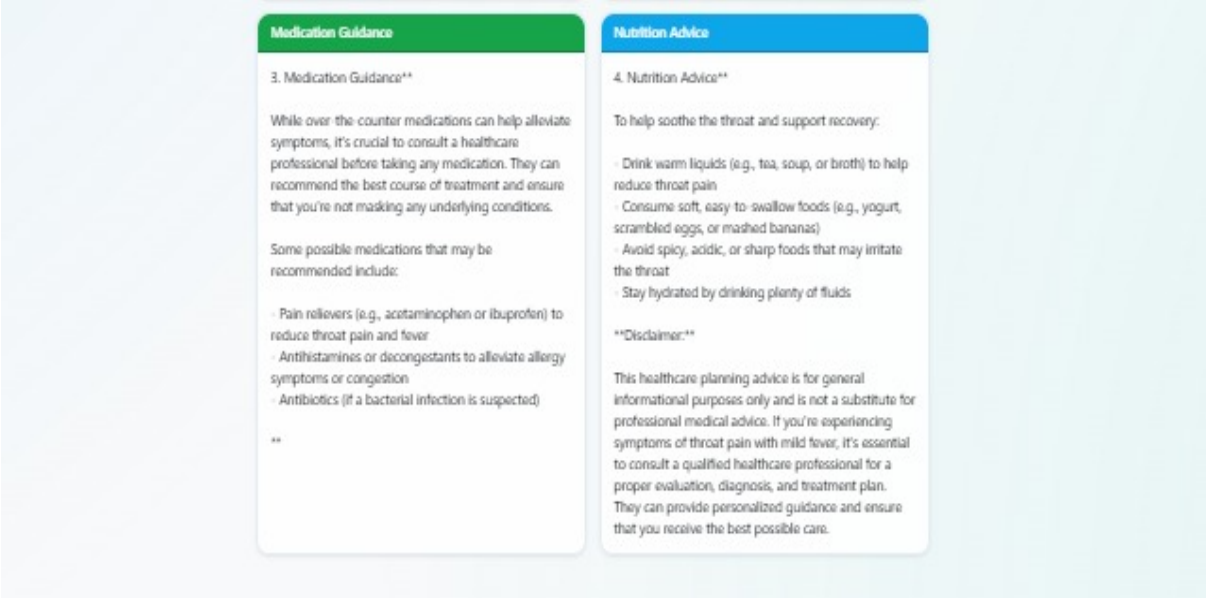
Once the AI model processes the request, the application receives the generated response and organizes it into four structured sections. These sections are displayed as informational cards on the webpage for better readability.

The four advisory sections include:

1. Possible Conditions
2. Precautions
3. Medication Guidance
4. Nutrition Advice

Each section provides useful information related to the symptoms entered by the user. The structured presentation ensures that users can easily understand the information without reading through large blocks of text.


Describe your symptoms
Symptoms*
Throat pain with mild fever
Age*
e.g. 35
Sex*
Select...
Duration (optional)
e.g. 2 days, 1 week
Model
Groq Llama
Generate advice
Possible Conditions
1. Possible Conditions**
Based on the symptoms described, several possible conditions could be contributing to the throat pain and mild fever. These may include:
- Common cold or flu
- Sore throat (pharyngitis)
- Tonsillitis (inflammation of the tonsils)
- Sinus infection
- Allergic reactions or postnasal drip
- Other viral or bacterial infections
It's essential to note that this is not an exhaustive list, and it's always best to consult a healthcare professional for a proper evaluation and diagnosis.
Red Flag Symptoms:
- Difficulty swallowing or breathing
- Severe throat pain or swelling
- High fever (over 103°F/39.4°C)
- Severe headache or stiff neck
- Confusion or disorientation
If you experience any of these symptoms, seek urgent or emergency care.
**
Precautions
2. Precautions**
To avoid exacerbating the condition or spreading the illness:
- Avoid close contact with others, especially those with weakened immune systems (e.g., young children, older adults, or people with chronic illnesses)
- Practice good hygiene: wash hands frequently, avoid touching the face, and clean surfaces and objects regularly
- Stay hydrated by drinking plenty of fluids (e.g., water, clear broths, or electrolyte-rich beverages)
- Get plenty of rest to help your body recover
**


Medication Guidance
3. Medication Guidance**
While over-the-counter medications can help alleviate symptoms, it's crucial to consult a healthcare professional before taking any medication. They can recommend the best course of treatment and ensure that you're not masking any underlying conditions.
Some possible medications that may be recommended include:
- Pain relievers (e.g., acetaminophen or ibuprofen) to reduce throat pain and fever
- Antihistamines or decongestants to alleviate allergy symptoms or congestion
- Antibiotics (if a bacterial infection is suspected)
**
Nutrition Advice
4. Nutrition Advice**
To help soothe the throat and support recovery:
- Drink warm liquids (e.g., tea, soup, or broth) to help reduce throat pain
- Consume soft, easy-to-swallow foods (e.g., yogurt, scrambled eggs, or mashed bananas)
- Avoid spicy, acidic, or sharp foods that may irritate the throat
- Stay hydrated by drinking plenty of fluids
Disclaimer:
This healthcare planning advice is for general informational purposes only and is not a substitute for professional medical advice. If you're experiencing symptoms of throat pain with mild fever, it's essential to consult a qualified healthcare professional for a proper evaluation, diagnosis, and treatment plan. They can provide personalized guidance and ensure that you receive the best possible care.

3.1.4 Error Handling Output

The system also handles cases where incorrect or incomplete input is provided. For example, if the user submits the form without entering symptoms or provides invalid age values, the system displays an appropriate error message.

This feature ensures that the application remains robust and prevents incorrect data from being processed.

3.1.5 Model Selection Feature

Another feature demonstrated in the output is the ability to choose between different AI model providers. The user can select either the Gemini model or the Groq model before submitting the form.

This flexibility allows the system to work with multiple AI providers and ensures better adaptability in case one service is unavailable.

3.2 REPORTS / DASHBOARDS / MODELS

Since the Healthcare Planning Assistant is designed as a lightweight web-based advisory system, it does not include traditional dashboards or analytical reports. However, the system effectively utilizes external AI models to generate structured healthcare guidance based on user input.

The application integrates with two different Large Language Model providers:

Google Gemini API
The Gemini model is used to analyze the user's symptoms and generate informative healthcare advisory content. The model processes the prompt template provided by the backend and returns a detailed response in natural language.

Groq LLM API
The Groq model serves as an alternative AI provider and performs similar analysis on the input data. The ability to switch between models demonstrates the modular design of the system.

The application uses LangChain to create prompt templates and manage interactions with these AI models. LangChain simplifies the process of sending input variables such as symptoms, age, gender, and symptom duration to the AI model.

After receiving the response from the AI model, the application parses the generated text and extracts structured sections that are presented to the user.

Although the system currently does not generate statistical reports or dashboards, it demonstrates the practical use of AI models in generating intelligent advisory outputs. Future versions of the system could include analytics dashboards, user history tracking, and health trend analysis.

3.3 KEY OUTCOMES

The development and implementation of the Healthcare Planning Assistant resulted in several important outcomes that demonstrate the effectiveness of the system.

1. **Successful Integration of AI Models**

   One of the key outcomes of the project is the successful integration of Large Language Models into the web application. The system is able to communicate with external AI providers and generate meaningful healthcare guidance based on user input.

2. **Efficient Data Processing and Validation**

   The use of Pydantic models ensures that user inputs are validated before processing. This improves the reliability of the application and prevents incorrect or incomplete data from being sent to the AI models.

3. **Structured Advisory Output**

   The system successfully organizes the generated healthcare guidance into clearly defined sections. This structured presentation improves readability and allows users to quickly understand the information provided by the application.

4. **User-Friendly Interface**

   The application interface is simple and intuitive, enabling users to easily enter their symptoms and receive results. The use of Bootstrap styling enhances the visual layout and responsiveness of the system.

5. **Modular and Scalable Architecture**

   The project demonstrates a well-organized backend architecture with clearly separated components such as routing, service layers, configuration management, and AI model clients. This modular design allows the system to be easily extended with additional features in the future.

6. **Real-World Application of AI Technologies**

   The project showcases how artificial intelligence can be integrated into real-world applications to provide useful assistance to users. By combining web technologies

with AI services, the system demonstrates the potential for building intelligent healthcare support platforms.

7. **Improved Healthcare Awareness**

   Although the application does not replace professional medical consultation, it provides useful preliminary information that can help users better understand their symptoms and encourage them to seek medical advice when necessary.

4. CONCLUSION

The Healthcare Planning Assistant project demonstrates the development of an intelligent web-based application that provides structured healthcare guidance based on user-reported symptoms and demographic information. The system integrates modern web development technologies with Artificial Intelligence models to create a platform that assists users in understanding potential health conditions and recommended precautions.

The project was developed to address the growing need for accessible and user-friendly health information systems. Many individuals search for symptom-related information online but often encounter scattered, complex, or unreliable sources. This application simplifies that process by allowing users to input their symptoms and receive organized healthcare advisory responses in a structured format.

The system uses the FastAPI framework to build a high-performance backend capable of handling user requests efficiently. The frontend interface is developed using Jinja2 templates and Bootstrap, ensuring a responsive and easy-to-use user experience. By integrating LangChain, the application communicates with Large Language Models such as Google Gemini and Groq, which generate healthcare-related guidance based on user input.

A major achievement of the project is the implementation of a modular architecture that separates functionalities into well-defined layers, including the presentation layer, web/API layer, service layer, and configuration layer. This structured design improves maintainability, scalability, and overall clarity of the system.

The system also validates user inputs using Pydantic models, ensuring that incorrect or incomplete data is handled appropriately. After validation, the system sends the relevant information to the selected AI model and organizes the generated response into clearly defined sections such as Possible Conditions, Precautions, Medication Guidance, and Nutrition Advice, making the output easy for users to understand.

Another important feature of the system is its ability to support multiple AI providers. Users can choose between different AI models, which increases flexibility and ensures the system can continue functioning even if one provider becomes unavailable. This design also allows future integration with additional AI services.

Although the system does not replace professional medical consultation, it acts as a supportive informational tool that helps users better understand their symptoms and encourages them to seek appropriate medical advice when necessary. The project demonstrates how Artificial Intelligence can be applied in healthcare-related applications to provide preliminary insights and improve health awareness.

Overall, the Healthcare Planning Assistant successfully achieves its objectives by creating a functional AI-powered healthcare advisory system. It also establishes a strong foundation for future enhancements and highlights the potential of combining web technologies with artificial intelligence to develop intelligent healthcare assistance platforms.

5 .FUTURE SCOPE & ENHANCEMENT

The Healthcare Planning Assistant project demonstrates how artificial intelligence and modern web technologies can be combined to provide preliminary healthcare guidance to users. While the current implementation provides a functional and efficient system, there are several opportunities for future improvements that can enhance the effectiveness, scalability, and usability of the application.

To further expand the capabilities of the system, the following future scope areas along with their key enhancements can be considered:

1. Database Integration

Future versions of the system can include a database to store user data and advisory responses. This would allow users to access their previous health queries and maintain a record of symptom history.

Key Enhancements:

1. Storage of user symptom history and advisory reports.
2. Ability for users to review past health queries.
3. Analysis of symptom patterns and health trends over time.

2. User Authentication and Profile Management

Introducing a user login and profile system would allow the application to offer a more personalized experience.

Key Enhancements:

1. User account creation and secure login functionality.
2. Personalized health records and saved advisory responses.
3. Customized recommendations and follow-up reminders.

3. Integration of Additional AI Models and Healthcare APIs
Expanding AI integration can improve the accuracy and reliability of healthcare guidance provided by the system.

Key Enhancements:

1. Support for additional advanced AI models.
2. Integration with verified medical knowledge bases or healthcare APIs.
3. Improved quality and credibility of generated healthcare advice.

4. Analytics Dashboards and Reporting Tools
Developing analytics features can help analyze system usage and health-related trends.

Key Enhancements:

1. Dashboards displaying frequently reported symptoms.
2. Visualization of common health concerns and usage statistics.
3. Insights useful for developers, researchers, and administrators.

5. Enhanced User Interface and Accessibility
Improving the user interface can make the system more interactive and accessible to a wider audience.

Key Enhancements:

1. Voice-based symptom input.
2. Chatbot-style conversational interaction.
3. Mobile-responsive design and dedicated mobile applications.

6. Multilingual Support
Expanding language support can significantly increase the reach of the application.

Key Enhancements:

1. Support for multiple languages in both input and responses.
2. Improved accessibility for users from diverse linguistic backgrounds.
3. Broader global usability of the platform.

7. Custom Machine Learning Models
Future development could include machine learning models specifically trained on medical datasets.

Key Enhancements:

1. Development of domain-specific healthcare models.

2. More accurate and specialized health insights.
3. Reduced reliance solely on external language models.

8. Security and Privacy Improvements
As the system evolves, ensuring strong data protection will become increasingly important.

Key Enhancements:

1. Stronger data encryption mechanisms.
2. Secure authentication and access control.
3. Compliance with healthcare data protection standards.

9. Integration with Healthcare and Telemedicine Services
The system can eventually connect users with real healthcare providers for further medical consultation.

Key Enhancements:

1. Direct connection with doctors or healthcare professionals.
2. Integration with telemedicine platforms.
3. Transformation into a comprehensive digital healthcare support system.